

AgentDescent: Gradient Descent Where the Parameters Are Agents

A Parallel, Asynchronous, Merge-Based Framework for Self-Evolving Agent Artifacts

Birfy Chen

birfychen@gmail.com

<https://github.com/Birfy/agentdescent>

August 2026

Abstract

Self-improving agent systems — agents that evolve their own skills, prompts, and harnesses — are today overwhelmingly *serial*: one candidate is proposed, evaluated, and accepted or rejected per iteration, bounding improvement throughput at $O(1/T_{\text{iter}})$. We present **AgentDescent**, a framework that transplants the deep-learning *training stack* onto agent self-evolution. The “parameters” are a library of evolvable artifacts (skills, prompts, harness modules, verifiers); the “gradients” are *diffs carrying evidence cards*; the “optimizer step” is a *merge decision* over a git-backed, version-vectored ledger. N workers propose diffs in parallel and a barrier-free asynchronous aggregator merges them, targeting $O(N/T_{\text{iter}})$ improvement throughput. The one place the analogy must break defines the system: *gradients add, diffs do not* — so aggregation is not averaging but conflict resolution, statistical acceptance under a per-artifact Beta posterior, and transactional commit. Three execution concepts carry the design: *parallelism* (synchronous data-parallel rounds, whose barrier waits on the slowest rollout), *asynchrony* (a barrier-free runtime in which staleness becomes a first-class, per-diff lag η governed by a version-lag budget and by Full / Guarded / Reflective policies), and *reflective merge* (when concurrent proposals contradict on a single key, a model writes their union and the acceptance gate — not the model — decides). A layered governance model (frozen safety layer, oracle-gated harness layer, freely merging skill layer) keeps the self-referential loop from polluting itself. We demonstrate generality by porting *eighteen* published self-evolution algorithms — seven benchmark-faithful (ACE, GEPA, EvoSkill, SkillOpt, ADAS, DGM, OpenEvolve) and eleven declared microports and analogues — all running unchanged under serial, synchronous, and barrier-free asynchronous scheduling. On a live model (GEPA/HotpotQA), removing the round barrier lifts effective concurrency from $1.85\times$ to $3.22\times$ over a true serial control at a pinned rollout budget; an equal-budget comparison of merge-of- N against best-of- N forking is reported honestly as not yet separated, and we analyze why. Code, all eighteen ports, and every measured number’s reproduction command are open source.

1 Introduction

A rapidly growing literature shows that large-language-model agents can improve their own artifacts: their prompts [1, 4], their in-context playbooks [30], their skill libraries [22, 32], their workflows [7, 29], and even their own code [15, 16, 26, 28]. Nearly all of these systems share an execution shape inherited from their conceptual ancestor, the Gödel machine [18]: a *serial* improvement loop. One candidate modification is proposed, evaluated against a benchmark, and accepted or rejected; then the next. If one iteration takes T_{iter} wall-clock seconds, improvement throughput is bounded at one accepted change per T_{iter} — regardless of how much compute is available.

Deep learning faced the same bottleneck a decade ago and solved it with a now-standard stack: data-parallel workers computing gradients concurrently, parameter servers aggregating them [11], lock-free and staleness-tolerant asynchronous updates [14], adaptive per-parameter step sizes [10], and gradient surgery when updates conflict [27]. The premise of this paper is that the same stack, suitably reinterpreted, applies to agent self-evolution.

AgentDescent realizes this premise (Fig. 1). The parameter tensor becomes a *library of evolvable artifacts*; the gradient becomes a *diff* annotated with an *evidence card* recording the rollout that motivated it; the parameter server becomes a git-backed, version-vectored *ledger*; and the optimizer step becomes an *aggregator* that merges concurrent diffs. N workers roll out tasks against a snapshot of the library and propose diffs in parallel; a barrier-free asynchronous merger aggregates them into a shared, version-controlled artifact library, targeting $O(N/T_{\text{iter}})$ improvement throughput where the serial loop is bounded at $O(1/T_{\text{iter}})$.

The analogy is productive precisely because of where it breaks. *Gradients add; diffs do not*. Two gradients from different minibatches can always be summed; two textual edits to the same artifact may be complementary, redundant, or contradictory. Aggregation in AgentDescent is therefore not averaging but a discrete-space optimizer step: staleness filtering, conflict resolution, fusion of complementary edits, a statistical acceptance test under a per-artifact Beta posterior, and a transactional compare-and-swap commit (Section 4). A second disanalogy is equally load-bearing: training code is not modifiable by the gradients it computes, but a self-evolving agent system *can* rewrite its own verifier — so AgentDescent enforces a layered governance model in which safety-critical components are frozen and high-blast-radius artifacts are forced through an oracle (Section 6).

Contributions.

1. **A systems design** that maps the deep-learning training stack — parameter server, asynchronous SGD, staleness bounds, adaptive per-parameter tests, gradient surgery, weight averaging, EMA — onto parallel agent self-evolution, with an explicit account of where the analogy holds and where it must break (Section 3).
2. **A discrete-space aggregator** implementing merge as optimization: staleness policies (Full / Guarded / Reflective) over a per-diff lag measure η , PCGrad-style conflict projection, model-soup-style fusion with an optional fusion tournament, Beta-posterior acceptance with annealed confidence, and dual-branch (dev/stable) promotion (Section 4).
3. **Two execution runtimes and three parallelism paradigms**: a synchronous data-parallel round loop and a barrier-free asynchronous stage-orchestration runtime with a version-lag budget (`async_ratio`) and backpressure; plus data-, tensor-, and pipeline-parallel decompositions of the artifact library (Section 5).
4. **Generality, demonstrated**: eighteen published self-evolution algorithms ported onto one engine — seven reproducing their paper’s own benchmark, eleven as declared mechanism microports and analogues — each runnable, offline-tested, and measured under all three schedulers (Section 7).
5. **An honest empirical study** (Section 8): near-linear dispatch scaling to 8 workers; $5.8\times$ thread-level speedup on real API rollouts; $1.85\times \rightarrow 3.22\times$ effective concurrency from removing the round barrier on a live model at a pinned budget; a staleness-policy and lag-budget characterization; and an equal-budget merge-vs-fork comparison that we report as *not yet separated*, together with the diagnosis of why separation is hard at small budgets.

Table 1: The central analogy. Each row of the training stack has a concrete counterpart in AgentDescent; the final row is a *disanalogy* that the design enforces rather than papers over.

Model training	AgentDescent (parallel self-evolution)
parameter tensor θ	library of Evolvable artifacts
gradient g	Diff + EvidenceCard
parameter server	git-backed, version-vectored Ledger
optimizer step	Aggregator merge decision
per-parameter adaptive LR (Adam)	per-artifact Beta-posterior acceptance test
staleness / decoupled PPO	per-diff lag η + rebase re-verification
partial rollout	straggler detection (ResumeQueue)
EMA / weight averaging	stable/dev dual branch
training code (not self-modifiable)	L0 frozen governance layer

2 Related Work

Self-evolving agents. Prompt evolution spans PromptBreeder’s self-referential mutation [4] and GEPA’s reflective Pareto search [1]; context/playbook evolution is exemplified by ACE [30]; skill-library growth by Voyager [22], SkillWeaver [32], and EvoSkill-style automated skill discovery; workflow and harness search by AFlow [29] and ADAS [7]; self-modifying code by the Darwin Gödel Machine [28], SICA [16], and Gödel Agent [26]; evolutionary program search by AlphaEvolve [15] and its open reimplementation OpenEvolve [19]; and label-free self-play by Absolute Zero [31], R-Zero [8], and Agent0 [24]. Reflection-based refinement [12, 20] supplies the inner proposal mechanism many of these share. Almost all publish a serial outer loop; AgentDescent treats that loop as one scheduling choice among three.

Parallel and asynchronous training systems. The parameter server [11], lock-free asynchronous SGD [14], and bounded-staleness asynchronous RL systems such as AReaL [5] and ROLL Flash [17] provide the systems vocabulary AgentDescent reuses: staleness bounds, decoupled actors and learners, and lag budgets. Adam’s per-parameter adaptivity [10], PCGrad’s conflict projection [27], and model soups [23] are the optimizer-level analogies for the per-artifact acceptance test, conflict resolution, and fusion respectively. Tensor and pipeline parallelism [9, 21] motivate the corresponding artifact-space decompositions. Concurrent work on parallelizing self-evolution (FlashEvolve-style barrier-free evolution, CoEvoSkills’ co-evolutionary verification [3]) indicates the throughput premise is a live hypothesis rather than community consensus; AgentDescent’s contribution is a narrow, defensible engineering synthesis — concurrent, staleness-bounded, conflict-resolved *diff-level merge* over a git-backed versioned ledger — with the measurement harness to test it.

3 The Training-Stack Analogy and System Design

3.1 Evolvables, diffs, and evidence cards

An Evolvable is any versioned artifact the system may modify: a skill document, a prompt, a playbook of bullets, a harness module, a directory of files, or program code. A Diff is a keyed set of operations against one or more evolvables — a *strategy* (e.g., AppendRules, KeyedRules, a per-bullet playbook, or file-path-keyed directory edits) maps a natural-language proposal into diff operations, which is what makes distinct edits fusible and conflicting edits detectable. Every diff carries an EvidenceCard: the task, the rollout output, the reward, and the base version the proposing worker observed. The card is the unit of

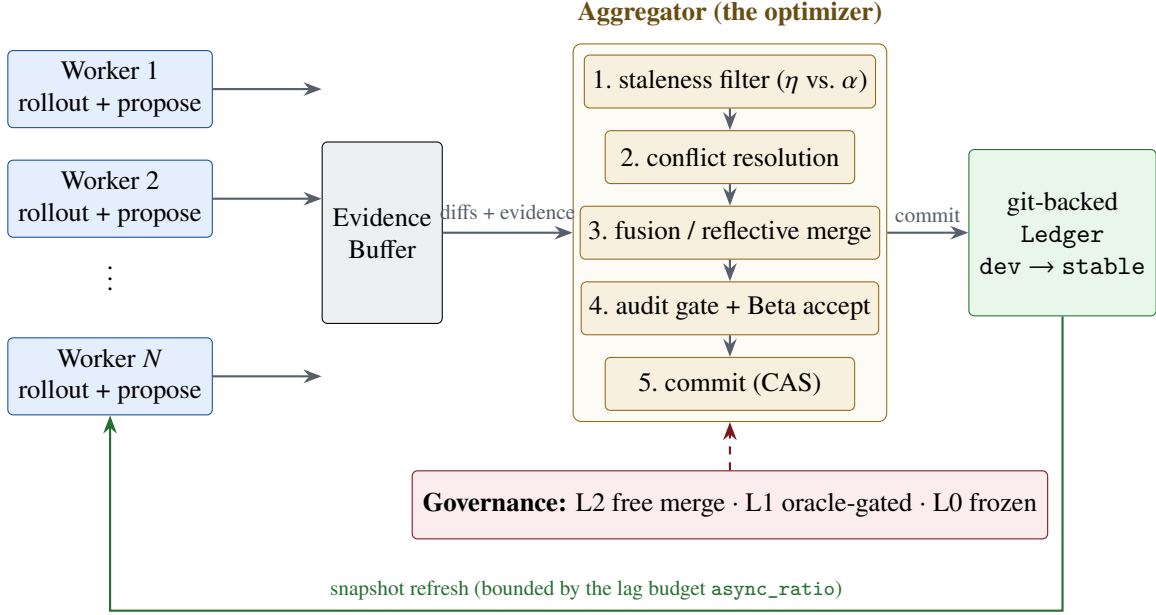


Figure 1: AgentDescent architecture. Evidence flows left to right: N workers roll out tasks against ledger snapshots and emit diffs with evidence cards into a thread-safe buffer; a dedicated aggregator thread runs the five-stage merge pipeline and commits winners to the git-backed ledger. The green return path is the only feedback edge: workers refresh their snapshot from the ledger, and the lag budget `async_ratio` bounds how far behind head a snapshot may drift. In the synchronous runtime a round barrier sits between the two sides; in the barrier-free runtime they never wait for each other. Governance (dashed) forces high-blast-radius merges through an oracle and freezes the safety layer.

statistical bookkeeping; discarded diffs settle their evidence back into the pool so information is not lost with the proposal.

3.2 The ledger

The Ledger is the parameter server: a git-backed store with per-artifact version vectors, compare-and-swap commits, and a dual `dev/stable` branch pair. `dev` receives merges; `stable` is promoted only after K regression-free rounds on `dev` — an EMA-style confirmation in which a new commit *restarts* the clock, so the artifact most likely to be promoted is the one that stopped changing because nothing beat it. Workers read snapshots; the version vector is what makes per-diff staleness η computable at merge time.

3.3 The one required disanalogy

Because diffs do not add, every downstream design choice follows from replacing summation with a decision procedure. Averaging two contradictory edits is not meaningful; ranking and projecting them is. This is why the aggregator (next section) is best understood not as a reducer but as a discrete-space optimizer whose “step” is a merge decision, and why acceptance is statistical — a per-artifact posterior over whether the candidate improves held-out reward — rather than a learning rate.

4 The Aggregator as a Discrete-Space Optimizer

Evidence cards are bucketed by artifact. When a bucket triggers (batch size B , or a timeout $T_{\max}$ so cold artifacts do not starve), the aggregator runs one optimizer step (Fig. 1, center):

1. **Staleness filter.** Each diff’s lag is $\eta = \max(\text{head} - \text{base})$ over the artifacts it touches. $\eta = 0$ proceeds; $0 < \eta \leq \alpha$ is *rebased and cheaply re-verified* (does the claimed delta still hold on the new head?); $\eta > \alpha$ is discarded, its evidence settled back into the pool. The bound α adapts to artifact heat, and contract-breaking diffs force $\alpha = 0$. This is the bounded-staleness discipline of asynchronous RL systems [5] applied per-diff.
2. **Conflict resolution.** Syntactic conflicts (hunk overlap) and semantic conflicts (contradictory operations) are detected; contradictions are projected out PCGrad-style [27], keeping the better of the pair as measured on a shared held-out subset.
3. **Fusion.** Complementary surviving diffs are fused — the model-soup analogy [23] — and the union proceeds to the gate. An optional *fusion tournament* ranks the union against the individual candidates on held-out data first (Section 4.1).
4. **Audit gate.** The merge decision is itself submitted to an audit scheduler: high-blast-radius or low-trust merges are forced through the oracle, which can veto outright *before* the acceptance test runs. The optimizer audits itself.
5. **Statistical acceptance.** A candidate commits only if $P(\Delta > 0) > 1 - \delta$ under a per-artifact Beta posterior over held-out task outcomes — a posterior test rather than a point threshold, playing the role Adam’s per-parameter adaptivity plays in training [10]. The confidence requirement δ anneals with artifact version (the learning-rate-decay analogue), and a trust region caps diff size.
6. **Commit.** Compare-and-swap on dev, one artifact per merge.
7. **Dual-branch promotion.** `dev` $\rightarrow$ `stable` after K regression-free rounds, as above.

4.1 Does merging average the improvements away?

The sharpest objection to merge-based evolution is that two individually good local edits may be jointly worse. Two properties bound the damage. First, *the acceptance gate, not the tournament, is the safeguard*: the gate scores the fused candidate on the full held-out set and refuses a measured regression, so a harmful fusion never commits. Second, the fused union is a *superset* of every individual diff, so committing it loses no proposal — it merely carries some that looked negative this round, and the next round proposes from there. The only case the tournament decides that the gate does not is a fusion that beats the base artifact but loses to one of its singles; that case is recoverable, which is why the tournament is off by default (it costs one held-out sweep per candidate, unconditionally, against a conditional and recoverable gain). It remains available (`fusion_tournament=True`) because it is the only instrument that can *measure* the objection: it reports a win rate, mean gain, the losing tail, and — critically — a *contested* counter for tournaments where a fused candidate was actually built and ranked. We deliberately publish no headline win rate: the quantity is a property of the artifact’s key space and of proposal overlap, not of the mechanism, and would not transfer across datasets (Section 8.5 reports it in situ for one workload).

5 Parallelism, Asynchrony, and Reflective Merge

This section develops the framework’s three central execution concepts in order of increasing subtlety: what parallelism buys and where its ceiling is; what removing the barrier costs and how that cost is governed; and what to do when concurrent proposals *contradict* rather than compose.

5.1 Parallelism: the round barrier and its ceiling

The baseline runtime is **synchronous data parallelism**: every worker holds the same ledger snapshot, rolls out a shard of tasks, and proposes diffs; a round barrier waits for *all* workers; then one aggregator step merges the round’s diffs; then the next round begins. It is deterministic and easy to reason about, and because agent rollouts are I/O-bound (the worker spends its time waiting on a model), plain threads already deliver real overlap — measured, eight threads speed up real API rollouts 5.8× while pure-CPU work gains nothing (1.1×).

The barrier, however, imposes a structural ceiling that no worker count can lift. A round lasts as long as its *slowest* rollout — wall-clock per round is $\mathbb{E}[\max \text{ of } N]$ rather than $\mathbb{E}[\text{latency}]$ — and modern reasoning models have exactly the heavy-tailed latency that maximizes this gap: a short answer and a 2,000-token deliberation are the same call. Between rounds, a second serial region appears: the aggregator’s held-out evaluations run while every worker idles. Both costs are inherent to the *schedule*, not the workload, which motivates removing the barrier.

5.2 Asynchrony: barrier-free evolution, staleness, and the lag budget

The **barrier-free runtime** deletes the round structure entirely. Worker threads continuously roll out and propose; a dedicated aggregator thread continuously drains the evidence buffer and merges; nobody waits for anybody (Fig. 1). This is the partial-rollout, no-barrier discipline of asynchronous RL systems [5, 17] ported to self-evolution: the rollout/propose stage and the merge/commit stage *overlap* instead of alternating, so a slow rollout delays only its own diff.

Asynchrony’s price is **staleness**: a worker may propose against a snapshot the ledger has already moved past. AgentDescent makes staleness a first-class, per-diff quantity — $\eta = \max(\text{head} - \text{base})$ over the artifacts the diff touches, computable because the ledger is version-vectored — and bounds it with two independent mechanisms. First, the **lag budget** `async_ratio` caps drift at the source: a worker refreshes its snapshot once head is more than `async_ratio` versions past it, and a backpressure signal forces a global resync if evidence keeps arriving while nothing commits. Small budget $\rightarrow$ near-synchronous behavior; large budget $\rightarrow$ high throughput and many stale diffs. Second, a **staleness policy** decides what to do with the stale diffs that arrive anyway, answering ACCEPT/REBASE/DISCARD from each diff’s (η, α) :

- **Full** — use stale diffs directly, η ignored: maximum throughput, minimum safety. Sound only where diffs compose regardless of base version.
- **Guarded** — version-gated bounded staleness in the spirit of AReal [5]: accept $\eta=0$, rebase $\eta \leq \alpha$, discard beyond. Safe, but it *pays for safety in discarded work* — at a large lag budget on our reference domain it throws away ~85% of its rollouts (Fig. 4).
- **Reflective** — always rebase onto the current head and *re-verify*: cheaply re-check whether the diff’s claimed improvement still holds there, and discard only if it does not. The insight is that a stale diff is not a wrong diff — it is an unproven one — so the policy converts would-be-discards back into candidates, recovering most of Guarded’s waste for the price of a cheap re-check.

One footgun deserves a paragraph because it generalizes: the lag budget is denominated in artifact *versions*, so its right value depends entirely on how much wall-clock a version represents. The default (3) is tuned for seconds-long model rollouts; on a domain where a rollout costs microseconds, a worker drifts past any budget almost instantly — measured, `async_ratio=3` there discards 93% of its evidence and never reaches the quality bar, while 1 comes within a point of the synchronous path. A run that discards more than half its evidence now warns at runtime.

5.3 Reflective merge: when a dictionary update cannot merge

The fusion stage’s default mechanism is a keyed union: a strategy maps each proposal into diff operations on keys (playbook bullets, rule slots, file paths), and *distinct* keys fuse for free — two workers editing different files, or appending different bullets, combine by construction. This is the model-soup analogy [23], and it silently assumes the artifact’s key space is fine-grained enough for concurrent proposals to land on different keys.

Many important artifacts violate that assumption. A prompt held in a *single* instruction slot — GEPA’s artifact shape [1], and arguably the common case for prompt evolution — makes every pair of concurrent proposals a write–write conflict on one key. The keyed machinery then has only bad options: a dictionary update (`ops.update()`) keeps the last rewrite and silently drops the rest, and conflict resolution keeps the better of the pair — which quietly reduces merge-of- N to per-round *best-of- N selection*. That is a real mechanism, but it is not merging, and a framework that reports it as merging would be measuring the wrong thing (the contested counter exists precisely so such runs cannot be misread).

Reflective merge (`reflective_merge`) is the escape hatch: when surviving proposals contradict on a key, a model is asked to *write their union* — one candidate text that preserves the intent of each proposal — and that written union goes to the acceptance gate like any other candidate. The gate still has the last word (a union that regresses held-out score does not commit), so the model’s synthesis is a proposal generator, not a decision maker. The same idea appears at two places in the pipeline under one name: as the fusion mechanism just described, and inside the *Reflective staleness policy* above, where “rebase and re-verify” is the temporal version of the same move — carry the proposal’s intent onto a changed base and let measurement decide. Reflective merge is also a *cost* lever: fusing a round’s N surviving diffs into one candidate means one held-out admission sweep instead of N , which is measurable as fewer model calls at equal budget (Section 8.2).

5.4 Beyond data parallelism

Two further decompositions mirror the training-stack taxonomy, included for completeness: **tensor parallel** [21] splits one hot artifact into disjoint sections so concurrent edits are conflict-free by construction, and **pipeline parallel** [9] treats a dependency chain of artifacts as stages, back-propagating blame for a downstream failure to the earliest failing upstream stage. Orthogonally, heavy-tailed rollout *durations* are met with duration-estimated LPT dispatch [6] and straggler detection, and heavy-tailed task *value* with UCB task selection [2] and an oracle-audit budget spent by blast radius $\times$ uncertainty/trust.

6 Governance by Blast Radius

Artifacts sort into layers automatically by a declared `blast_radius`: **L2 (fast)** — local skills and prompts, merged under the full asynchronous pipeline; **L1 (slow)** — harnesses and verifiers, with serialized in-flight changes, staged rollout, and every merge forced through the oracle; **L0 (frozen)** — the oracle itself, the audit budget, merge permissions, and safety constraints, read-only to the loop. The frozen layer is the enforced form of the training-stack disanalogy in Table 1: without it, the self-referential loop eventually pollutes itself (the canonical failure being a verifier that learns to pass itself). For directory-shaped artifacts the same layering yields three entry points — skill directories (L2), agent definitions (L1), and executed agent *code*, which runs behind a frozen test suite that the candidate cannot rewrite (pristine test files are overlaid after materialization, so weakening the tests at run time also fails).

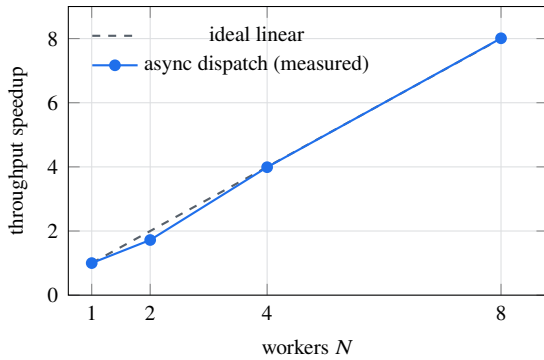


Figure 2: Rollout-dispatch throughput of the barrier-free runtime vs. worker count (fixed wall-clock window; `examples/efficiency.py`). Near-linear to 8 workers — the rollout stage holds no global lock. Across repeated runs the 8-worker figure landed between 7.8 $\times$ and 9.2 $\times$; the honest reading is “linear to within measurement noise at this scale.”

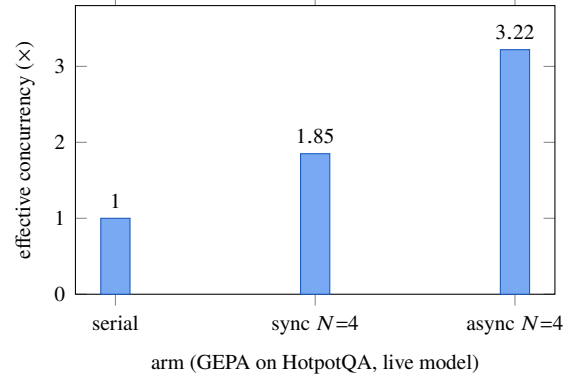


Figure 3: Effective concurrency (model-seconds $\div$ wall-clock) on a faithful GEPA port, HotpotQA, 16 rollouts pinned on every arm. The serial control is the published loop exactly (one worker, serial gate: overlap 1.00 $\times$). The round barrier is the whole difference between the parallel arms: the same four workers reach 1.85 $\times$ with it and 3.22 $\times$ without it. Wall-clock: 609 s / 239 s / 140 s.

7 Eighteen Algorithm Ports

A framework claiming generality should be measured by what it can express. Eighteen published self-evolution algorithms run on the engine — none bypasses it — via two routes: the seven *benchmark-faithful* ports are each a custom strategy and/or aggregator_factory plugged into `evolve()` (e.g., GEPA’s per-instance Pareto optimizer [1], ADAS’s keep-all archive [7], DGM’s archive-plus-selection [28], OpenEvolve’s MAP-Elites islands [13, 19]); the eleven *microports and analogues* are declarative policy definitions over one shared runner, so the runtime matrix can vary only the scheduler. Fidelity is to the *released code*, not just the paper (e.g., EvoSkill’s frontier is a top- K aggregate, not a per-instance Pareto front, and the port follows the code and says so); every port carries a fidelity class — *benchmark-faithful*, *mechanism microport*, *self-edit analogue*, *environment analogue*, or *inference analogue* — and analogues must not be cited as paper-benchmark reproductions. Every mechanism family in the field’s current taxonomy (evolutionary search, reflection, skill libraries, self-play, self-modifying code) is covered by at least one faithful port or one declared analogue. All eighteen run serial, synchronous-parallel, and barrier-free asynchronous without modification, because their optimizers keep shared state thread-safe.

8 Experiments

We report one *typical* result per claim rather than the full measurement tables, which live in the repository documentation together with the exact reproduction command for every number. All results are produced by scripts shipped in the repository; absolute times depend on host and endpoint, so ratios are the reproducible quantity. Live-model runs use `deepseek-v4-flash` or `GLM-5.2` through an OpenAI-compatible endpoint.

8.1 Parallelism: near-linear dispatch, a real barrier ceiling

Two measurements bracket what parallelism buys. *Dispatch* (Fig. 2): running the barrier-free runtime for a fixed wall-clock window and counting rollouts gives near-linear scaling through 8 workers — the $O(N/T_{\text{iter}})$ regime the design targets. *End-to-end*: a whole synchronous `evolve()` run at 8 workers reaches only 1.8 $\times$

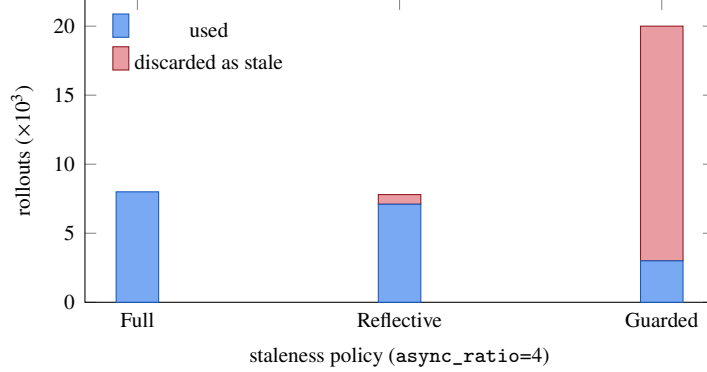


Figure 4: The three staleness policies at a deliberately large lag budget (examples/run_async). All three converge to the same final reward on the reference domain; what differs is the cost. Guarded discards ~85% of its rollouts as stale (~5.1 s wall-clock vs. ~3.2 s); Reflective recovers almost all of that would-be waste by rebasing and re-verifying instead of discarding. Ratios are the result — absolute counts scale with the host.

under uniform rollout latency and $1.7\times$ under a heavy-tailed, reasoning-model-like latency — the barrier ceiling predicted in Section 5.1, since a round waits for its slowest rollout and the inter-round gap is serial gate evaluation. On the same heavy-tailed workload, removing the barrier yields $2.65\times$ over the synchronous discipline: the barrier idles workers at 36–40% utilization waiting for the tail, while the barrier-free pipeline stays at ~100%.

8.2 The flagship: a live model against a true serial control

Stub latencies can flatter a scheduler, so the same question is put to a faithful port of a published serial algorithm on its own benchmark: GEPA on HotpotQA [1, 25], 16 rollouts pinned on every arm, and a control that is the upstream loop *exactly* — one worker *and* a serial gate, overlapping at precisely $1.00\times$ (a control that still evaluates concurrently is already partly parallel). Figure 3 shows the result: the synchronous arm reaches $1.85\times$ effective concurrency — consistent with the heavy-tail stub prediction — and removing the barrier takes the same four workers to $3.22\times$, with wall-clock falling $609 \rightarrow 239 \rightarrow 140$ s. Reflective merge also shows up here as the cost lever of Section 5.3: in a width-8 variant of the same setup, fusing each round’s surviving diffs into one candidate cut model calls by 28% relative to the serial arm (70 vs. 97), because N admission sweeps collapse into one. One caveat travels with the async arm: with no round boundary at which to stop, it overran the pinned budget (19 rollouts vs. 16), and at the default lag budget it discarded every stale proposal it saw — buying wall-clock and spending proposals.

8.3 Asynchrony’s cost structure: the staleness policies

Figure 4 makes the trade-off of Section 5.2 concrete: at a large lag budget all three policies converge to the same quality, and the entire difference is *what each one pays*. Guarded pays for safety in discarded work (~85% of rollouts thrown away); Full pays nothing on a domain whose diffs happen to compose; Reflective buys most of Guarded’s safety at a fraction of its waste, because rebase-and-re-verify converts would-be discards back into candidates. The lag-budget footgun of Section 5.2 was measured the same way: at `async_ratio=3` on a microsecond-rollout domain, 93% of evidence is discarded and the quality bar is never reached, while a budget of 1 comes within a point of the synchronous path.

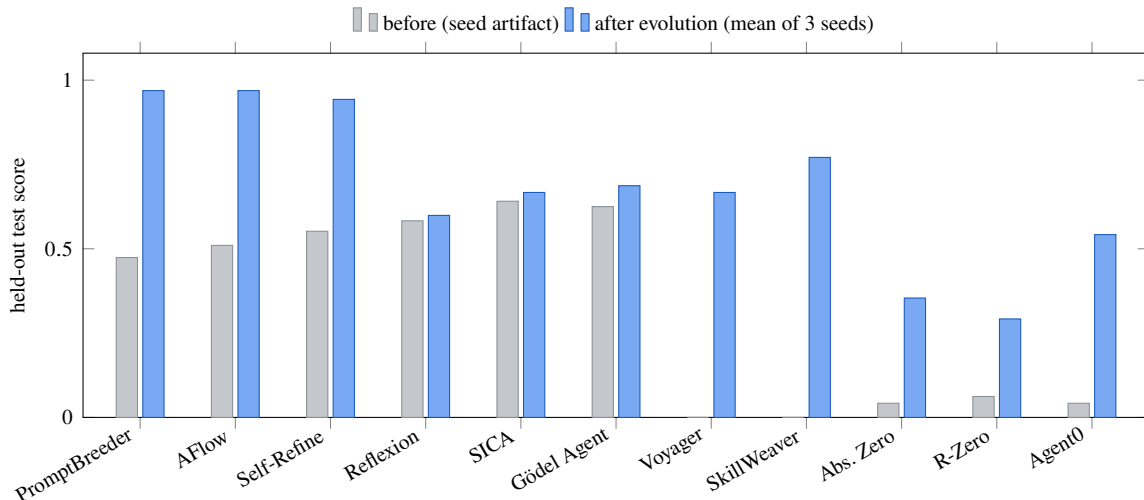


Figure 5: Held-out test score before and after evolution for the eleven microports and analogues, each run under the *most aggressive* configuration — barrier-free async pipeline, 8 workers, Full staleness, 80 rollouts, 3 seeds. All eleven improve. Between 3 and 10 of 240 proposals commit per method — the acceptance gate admits only strict held-out improvements, so sparse acceptance is the shape of the mechanism, not a low yield. Bars are comparable only within a domain: a gain is bounded by the headroom its baseline leaves.

8.4 Generality: the ports still learn under the aggressive schedule

Speed is worthless if the ports stop learning under the parallel schedulers, so quality is measured under the most aggressive one. Figure 5 shows the eleven declarative ports: all improve their held-out test metric, with gains bounded by each benchmark’s headroom exactly as they should be (the three GSM8K ports converge on 0.94–0.97 and separate on *cost*, not quality). The seven benchmark-faithful ports each run end to end on their own benchmark; three typical rows stand for the set: ACE lifts FiNER-139 validation 0.719 → 0.766 while curating a ten-bullet playbook, DGM’s best archived child fixes held-out vendored bugs at 0.906 against its seed agent’s 0.844 (its own solver growing from 18 to 79 lines), and OpenEvolve drives a function-minimization score from 0.964 to 1.4995 against a 1.5 ceiling. These single-seed runs establish that each port works, not paper-scale comparisons. Finally, the one-call entry point (`evolve_skill`) on 40 real HotpotQA rows lifts held-out exact match from 0.167 to 0.583, committing one proposal and rejecting three — the accepted one being the instruction “*Respond with only the requested answer, omitting any extra explanation or restatement.*”

8.5 Merge-of- N against best-of- N fork: an honest null

Every number above is a throughput number, and throughput cannot distinguish *merging* from *sampling-and-selecting*: N independent forks saturate N workers just as well. The distinguishing quantity is held-out quality at an *equal rollout budget*. On HotpotQA (three seeds, $N=3$, 9 rollouts pinned, reflective merge installed so contradicting proposals survive to be fused), merge-of-3 posts the highest median test score *and* the widest spread, the intervals overlap, and the framework’s own separation test refuses to call it a win — which is what that test is for. The diagnosis matters more than the verdict: at this budget the fused union was built *twice in the whole experiment*, because most merges saw only a single surviving diff — so this is not evidence that merging does not help; it is an experiment in which merging barely fired. It also confirms the single-key subtlety of Section 5.3 from the other side: without reflective merge, this workload’s merge arm would have been best-of- N selection wearing merging’s name. Larger budgets on multi-key artifacts are the

open measurement, and the baseline harness (serial / fork-with-oracle-and-selected / merge, budget-pinned, seeded) ships with the framework.

9 Scope, Limitations, and Honest Reporting

AgentDescent is a research reference implementation, not a production system, and several limits are stated rather than hidden. (i) The throughput premise is a *testable engineering hypothesis*, not community consensus; the equal-budget quality question (Section 8.5) is currently a null at small budgets. (ii) Turn-level checkpoint-and-resume of stragglers is not implemented, and neither is a stratified tail-canary held-out split. (iii) Most quality rows carry three seeds and the seven faithful ports carry one — evidence the mechanism runs and moves the metric it should, not paper-scale claims. (iv) Difficulty knobs calibrated on one model do not transfer: re-measured under a model swap at fixed configuration, the two ports whose difficulty was a knob (ACE’s top- k , SkillOpt’s hard-subset selector) lost their lift entirely — the mechanism ran correctly and there was nothing left to learn — while the ports whose difficulty is a property of the benchmark reproduced. (v) Several earlier headline numbers in the project’s own history were found to be irreproducible (no generating script) or measured with flattering definitions (setup time in the denominator, double-counted staleness denominators, one-run-per-arm comparisons), and were re-measured and corrected in place; every number in this paper is generated by a named script. We consider this correction trail part of the contribution: a framework for self-improving systems should hold its own evaluation to at least the standard its acceptance gate holds diffs to.

10 Conclusion

AgentDescent shows that the deep-learning training stack — parameter server, asynchronous data parallelism, bounded staleness, per-parameter adaptive acceptance, gradient surgery, weight averaging, and a frozen training substrate — has a coherent and implementable counterpart in agent self-evolution, once the one necessary disanalogy (diffs do not add) is taken seriously and merge is built as a discrete-space optimizer. Eighteen published algorithms run unchanged on the engine under serial, synchronous, and barrier-free scheduling; parallelism delivers the wall-clock it promises on live models (1.85× with a barrier, 3.22× without, against a true serial control); and the framework ships the instruments — fusion tournaments, contested counters, equal-budget baseline arms, separation tests — to answer the question that throughput cannot: whether merging is a new mechanism or an engineering convenience. On present evidence it is an engineering win with the quality question open, and the harness to close that question is now a `pip install` away.

Acknowledgments

The algorithm ports were contributed by chendanyang (the seven benchmark-faithful ports) and cyanneko (OpenEvolve and the eleven microports and analogues).

Reproducibility

All code is at <https://github.com/Birfy/agentdescent> (MIT license; `pip install agentdescent`; the core engine has zero required dependencies). Every measured number in this paper names its generating script: `examples/efficiency.py` (Sections 5.1 and 8.1), `bench/matrix_run.py` and `examples/gepa/gepa_prompt_evolution.py` (Section 8.2),

examples/run_async.py and bench/run.py (Section 8.3), bench/candidate_methods.py (Section 8.4), and bench/baselines_run.py (Section 8.5). Every port supports `--dry-run` (no API key, no model call) and carries an offline test suite; datasets are fetched from canonical sources by a dependency-free data layer. Absolute timings depend on host and endpoint; the repository documentation states, for each table, whether ratios or absolute values are the reproducible quantity.

References

- [1] Lakshya A Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J Ryan, Meng Jiang, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint arXiv:2507.19457*, 2025.
- [2] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2):235–256, 2002.
- [3] CoEvoSkills authors. Self-evolving agent skills via co-evolutionary verification. *arXiv preprint arXiv:2604.01687*, 2026.
- [4] Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*, 2023.
- [5] Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, et al. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning. *arXiv preprint arXiv:2505.24298*, 2025.
- [6] Ronald L Graham. Bounds on multiprocessing timing anomalies. *SIAM Journal on Applied Mathematics*, 17(2):416–429, 1969.
- [7] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- [8] Chengsong Huang, Wenhao Yu, Xiaoyang Wang, Hongming Zhang, Zongxia Li, Ruosen Fang, Jiaxin Huang, Tony Sanders, and Dong Yu. R-Zero: Self-evolving reasoning LLM from zero data. *arXiv preprint arXiv:2508.05004*, 2025.
- [9] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. GPipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in Neural Information Processing Systems*, 32, 2019.
- [10] Diederik P Kingma and Jimmy Ba. Adam: A method for stochastic optimization. *arXiv preprint arXiv:1412.6980*, 2014.
- [11] Mu Li, David G Andersen, Jun Woo Park, Alexander J Smola, Amr Ahmed, Vanja Josifovski, James Long, Eugene J Shekita, and Bor-Yiing Su. Scaling distributed machine learning with the parameter server. In *11th USENIX Symposium on Operating Systems Design and Implementation (OSDI 14)*, pages 583–598, 2014.
- [12] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *Advances in Neural Information Processing Systems*, 36, 2023.

- [13] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- [14] Feng Niu, Benjamin Recht, Christopher Ré, and Stephen J Wright. HOGWILD!: A lock-free approach to parallelizing stochastic gradient descent. *Advances in Neural Information Processing Systems*, 24, 2011.
- [15] Alexander Novikov, Ngân Vŭ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, et al. AlphaEvolve: A coding agent for scientific and algorithmic discovery. *arXiv preprint arXiv:2506.13131*, 2025.
- [16] Maxime Robeyns, Martin Szummer, and Laurence Aitchison. A self-improving coding agent. *arXiv preprint arXiv:2504.15228*, 2025.
- [17] ROLL Team. ROLL Flash: Accelerating RLVR and agentic training with asynchrony. *arXiv preprint*, 2025.
- [18] Jürgen Schmidhuber. Gödel machines: Self-referential universal problem solvers making provably optimal self-improvements. *arXiv preprint cs/0309048*, 2003.
- [19] Asankhaya Sharma. OpenEvolve: an open-source evolutionary coding agent. <https://github.com/codelion/openevolve>, 2025.
- [20] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36, 2023.
- [21] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-LM: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.
- [22] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *arXiv preprint arXiv:2305.16291*, 2023.
- [23] Mitchell Wortsman, Gabriel Ilharco, Samir Ya Gadre, Rebecca Roelofs, Raphael Gontijo-Lopes, Ari S Morcos, Hongseok Namkoong, Ali Farhadi, Yair Carmon, Simon Kornblith, et al. Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time. In *International Conference on Machine Learning*, pages 23965–23998, 2022.
- [24] Peng Xue, Cheng Wu, Zhen Lin, et al. Agent0: Unleashing self-evolving agents from zero data via tool-integrated reasoning. *arXiv preprint*, 2025.
- [25] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William W Cohen, Ruslan Salakhutdinov, and Christopher D Manning. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of EMNLP*, 2018.
- [26] Xunjian Yin, Xinyi Wang, Liangming Pan, Xiaojun Wan, and William Yang Wang. Gödel agent: A self-referential agent framework for recursive self-improvement. *arXiv preprint arXiv:2410.04444*, 2024.
- [27] Tianhe Yu, Saurabh Kumar, Abhishek Gupta, Sergey Levine, Karol Hausman, and Chelsea Finn. Gradient surgery for multi-task learning. *Advances in Neural Information Processing Systems*, 33:5824–5836, 2020.

- [28] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.
- [29] Jiayi Zhang, Jinyu Xiang, Zhaoyang Yu, Fengwei Teng, Xionghui Chen, Jiaqi Chen, Mingchen Zhuge, Xin Cheng, Sirui Hong, Jinlin Wang, et al. AFlow: Automating agentic workflow generation. *arXiv preprint arXiv:2410.10762*, 2024.
- [30] Qizheng Zhang, Changran Hu, Shubhangi Upasani, Boyuan Ma, Fenglu Hong, Vamsidhar Kamanuru, Jay Rainton, Chen Wu, Mengmeng Ji, Hanchen Li, et al. Agentic context engineering: Evolving contexts for self-improving language models. *arXiv preprint arXiv:2510.04618*, 2025.
- [31] Andrew Zhao, Yiran Wu, Yang Yue, Tong Wu, Quentin Xu, Yang Yue, Matthieu Lin, Shenzhi Wang, Qingyun Wu, Zilong Zheng, et al. Absolute zero: Reinforced self-play reasoning with zero data. *arXiv preprint arXiv:2505.03335*, 2025.
- [32] Boyuan Zheng, Michael Y Fatemi, Xiaolong Jin, Zora Zhiruo Wang, Apurva Gandhi, Yueqi Song, Yu Gu, Jayanth Srinivasa, Gaowen Liu, Graham Neubig, et al. SkillWeaver: Web agents can self-improve by discovering and honing skills. *arXiv preprint arXiv:2504.07079*, 2025.